Software Engineering Models and Methods

TLA$^+$: [1/4] Basics

assoc. prof. Karolis Petrauskas

Vilnius University
Department of Software Engineering
Institute of Computer Science

2026

Русский военный корабль

0LjQtNC4INC90LDRhdGD0Lk=

Outline

Structure of the TLA$^+$ Language

Example: Round Synchronization

Example: Connection Locks

Example: Brute-force Problems

TLA$^+$ Syntax in More Details

Common Modelling Tricks

Follow-up

References

TLA$^+$ Overview

- The TLA$^+$ language is designed by Leslie Lamport at Microsoft Research lab.
  He is also the author of LaTeX, Paxos, Lamport clocks, etc.
- It was designed to model concurrent and distributed systems.
  It is easy to model parallel as well as sequential algorithms.
- Designed to be used above the code level.
  Designer can chose any suitable abstraction level.
- Easy to use in engineering (not only in the research).

TLA$^+$ Overview | Used at

- Amazon, e.g. for verification of AWS algorithms, Newcombe (2014); Newcombe et al. (2015);
- Microsoft, e.g. a keynote at TLA$^+$ Community Event (2020), TLA+ at Microsoft to build planetary-scale systems, *https://www.youtube.com/watch?v=UVHnuxWZkvk*;
- Intel, e.g. Pre-RTL Formal Verification: An Intel Experience, Beers (2008);
- MongoDB, Extreme Modelling in Practice, Davis et al. (2020);
- Elasticsearch, `https://github.com/elastic/elasticsearch-formal-models`;
- ...

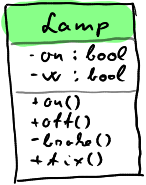
TLA$^+$ Overview | Basics

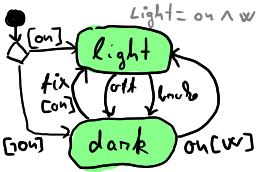
Lamport (2002)

- TLA stands for Temporal Logic of Actions (Lamport, 1994).
- TLA$^+$ is a specification language based on TLA.
- Specifications typically formulated for closed systems, but open systems are also supported (Abadi and Lamport, 1994).
  - Closed-system – specification describes the system and its environment.
  - Open-system – only the system is specified, including its preconditions.
- Encourages to use as little of temporal logic as possible.

  Because it is confusing if not trivial. So, mostly the predicate logic.
- A system is modelled as a state machine:
  - Structure of a state (a set of variables).
  - Initial state of the system (assignment of values to variables).
  - Actions – possible transitions in the system.

TLA$^+$ Overview | FSM



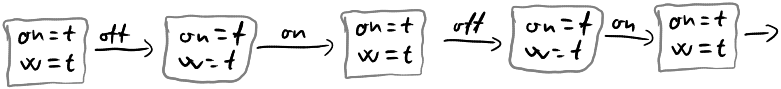


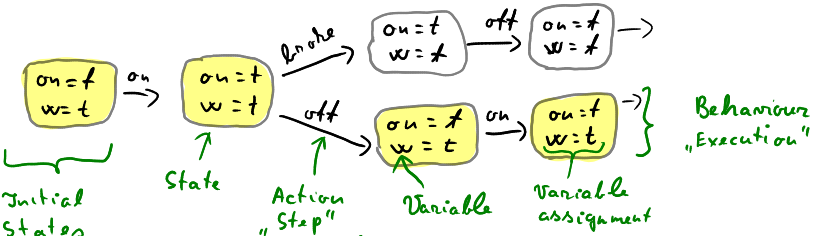
$Lamp = \langle I, N, T \rangle$

$I = \{0, 1\}$ — Initial st.

$N = \{\ldots\}$ — Transitions

$T = \emptyset$ — Terminal





TLA$^+$ Overview | Spec as a Formula

- The specification is written as a single formula in the following form:

$$Spec \triangleq Init \wedge \Box[Next]_{vars} \wedge Liveness \tag{1}$$

- Specification is typed in a plain text format, math symbols are mainly based on ASCII and LaTeX notation, e.g. `s \in S` stands for $s \in S$. The above formula would be written as:

```
Spec == Init /\ [][Next]_vars /\ Liveness
```

TLA$^+$ Overview | Modules

1. Specifications are organized in modules.
2. Each module has a set of constants (parameters of the spec);

   They are constant for entire execution, but can vary for different executions.
3. Variables (they are untyped);

   They change during an execution.
4. Formulas, written in the form *Name* $\triangleq$ *Definition*.

```
---------------- MODULE Example ------------------
EXTENDS Naturals
CONSTANT  C1
CONSTANTS C2, C3
VARIABLE  v1
VARIABLES v2, v3
vars == <<v1, v2, v3>>
Uniq == {v1, v2, v3}
Init == v1 = C1 /\ v2 = C2 /\ v3 = C3
Next == v1' = v1 - 1 /\ v2' = v2 + 1 /\ v3' = v3 + 2
--------------------------------------------------
Spec == Init /\ [][Next]_vars
==================================================
```

TLA$^+$ Overview | Properties

Specification can be used to verify properties of the algorithm:

1. Invariants – properties valid in each step, e.g. $\Box TypeOK$.
2. Temporal properties – e.g. $\Diamond\Box Completed$

```
TypeOK == v1, v2, v3 \in Nat
Inv    == v1 < v2 + v3
THEOREM Spec => [](TypeOK /\ Inv)
```

The properties can be verified in two ways:

a) Model-checked using TLC (for some models of the spec).
b) Proved using TLAPS (for all the possible models).

TLA$^+$ Overview | Tools

- **VSCode × "TLA+ (Temporal Logic of Actions)" plugin**.
  The ecosystem migrates to this IDE.
  - More convenient than the Eclipse based toolbox.
  - Java might be required additionally.
  - Supports TLC out of the box, TLAPS has to be compiled and enabled in the extension's settings.
- **CLI tools:**
  - **TLC** – model checker, can be used from the toolbox, or from the command line.
    E.g. for scripting, HPC.
  - **TLAPS** – the TLA$^+$ Proof System, uses other backends.
    Isabelle, Zenon, PTL, SMT solvers, etc.
  - **SANY** – TLA$^+$ parser and static analyser.
    Also handles the PlusCal translation.
  - **TLATEX** – produces PDF versions of the specifications.
- **Toolbox** – Eclipse based IDE, not maintained anymore.

TLA$^+$ Overview | Models and Model-checking (1/3)

A model is an instance of a specification with particular values for its constants, e.g.:

$$C1 \leftarrow 12, \quad C2 \leftarrow 20, \quad C3 \leftarrow 33. \tag{2}$$

1. A specification can have multiple models defined.
2. Models are used to model-check the specification.
3. Model-checking performs brute-force traversal over all possible states of the system and all the possible time interleavings of the events (actions).

TLA$^+$ Overview | Models and Model-checking (1/3)

A model is an instance of a specification with particular values for its constants, e.g.:

$$C1 \leftarrow 12, \quad C2 \leftarrow 20, \quad C3 \leftarrow 33. \tag{2}$$

1. A specification can have multiple models defined.
2. Models are used to model-check the specification.
3. Model-checking performs brute-force traversal over all possible states of the system and all the possible time interleavings of the events (actions).

4. It can be used to check:
   4.1 if the algorithm has a deadlock;
   4.2 if invariants are satisfied in every step;
   4.3 if temporal properties are satisfied in every execution.

TLA$^+$ Overview | Models and Model-checking (2/3)

Assume a specification:

$$Init \triangleq n = 0 \tag{3}$$

$$Next \triangleq n < 3 \wedge n' = n + 1 \tag{4}$$

$$Spec \triangleq Init \wedge \Box[Next]_n \tag{5}$$

TLA$^+$ Overview | Models and Model-checking (2/3)

Assume a specification:

$$Init \triangleq n = 0 \tag{3}$$

$$Next \triangleq n < 3 \wedge n' = n + 1 \tag{4}$$

$$Spec \triangleq Init \wedge \Box[Next]_n \tag{5}$$

A state in which a deadlock is reached:

$$[n = 3] \text{ with trace } [n = 0] \rightarrow [n = 1] \rightarrow [n = 2] \rightarrow [n = 3]. \tag{6}$$

TLA$^+$ Overview | Models and Model-checking (2/3)

Assume a specification:

$$Init \triangleq n = 0 \tag{3}$$

$$Next \triangleq n < 3 \wedge n' = n + 1 \tag{4}$$

$$Spec \triangleq Init \wedge \Box[Next]_n \tag{5}$$

A state in which a deadlock is reached:

$$[n = 3] \text{ with trace } [n = 0] \rightarrow [n = 1] \rightarrow [n = 2] \rightarrow [n = 3]. \tag{6}$$

An example of invariants of the specification:

$$TypeOK \triangleq n \in Nat, \quad RangeInv \triangleq 0 \leq n \wedge n \leq 3,$$
$$\text{LEMMA } Spec \Rightarrow \Box(TypeOK \wedge RangeInv). \tag{7}$$

TLA$^+$ Overview | Models and Model-checking (2/3)

Assume a specification:

$$Init \triangleq n = 0 \tag{3}$$

$$Next \triangleq n < 3 \land n' = n + 1 \tag{4}$$

$$Spec \triangleq Init \land \Box[Next]_n \tag{5}$$

A state in which a deadlock is reached:

$$[n = 3] \text{ with trace } [n = 0] \to [n = 1] \to [n = 2] \to [n = 3]. \tag{6}$$

An example of invariants of the specification:

$$TypeOK \triangleq n \in Nat, \quad RangeInv \triangleq 0 \leq n \land n \leq 3,$$
$$\text{LEMMA } Spec \Rightarrow \Box(TypeOK \land RangeInv). \tag{7}$$

An example of a temporal property:

$$Prop \triangleq \Diamond(n = 2), \qquad \text{LEMMA } Spec \Rightarrow Prop. \tag{8}$$

TLA$^+$ Overview | Models and Model-checking (3/3)

$$
\begin{array}{l}
\text{MODULE } \mathit{TLA_Lecture_Example_N} \\
\text{EXTENDS } \mathit{Naturals} \\
\text{CONSTANT } \mathit{MAX} \\
\text{VARIABLE } n \\
\mathit{Init} \triangleq n = 0 \\
\mathit{Next} \triangleq n < \mathit{MAX} \land n' = n + 1 \\
\mathit{Spec} \triangleq \mathit{Init} \land \Box[\mathit{Next}]_n \land \mathrm{WF}_n(\mathit{Next}) \\
\mathit{TypeOK} \triangleq n \in \mathit{Nat} \\
\mathit{RangeInv} \triangleq 0 \leq n \land n \leq \mathit{MAX} \\
\mathit{Prop} \triangleq \Diamond(n = \mathit{MAX} - 1) \\
\text{LEMMA } \mathit{Spec} \Rightarrow \Box(\mathit{TypeOK} \land \mathit{RangeInv}) \land \mathit{Prop}
\end{array}
$$

```
---------------------- MODULE TLA_Lecture_Example_N ----------
EXTENDS Naturals
CONSTANT MAX
VARIABLE n
Init == n = 0
Next == n < MAX /\ n' = n + 1
Spec == Init /\ [][Next]_n /\ WF_n(Next)
TypeOK   == n \in Nat
RangeInv == 0 <= n /\ n <= MAX
Prop     == <>(n = MAX-1)
LEMMA Spec => [] (TypeOK /\ RangeInv) /\ Prop
=============================================================
```

Outline

Structure of the TLA$^+$ Language

Example: Round Synchronization

Example: Connection Locks

Example: Brute-force Problems

TLA$^+$ Syntax in More Details

Common Modelling Tricks

Follow-up

References

How to share a secret?

- I have a secret S (let's say a number).
- How to give it to two persons so that they can recover the secret S only if both are participating?

How to share a secret?

- I have a secret S (let's say a number).
- How to give it to two persons so that they can recover the secret S only if both are participating?

Possible solution:

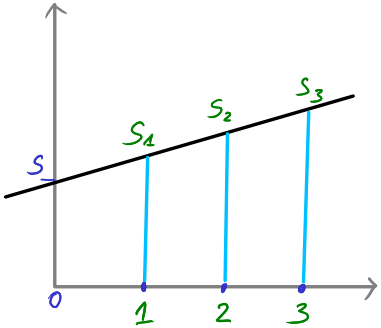
1. Choose random number A.
2. Give A to the first person
   and $B = S - A$ to the second.
3. The persons can add A and B to get S.

Example: Round Synchronization

How to share a secret so that it can be recovered by any 2 of 3 participants?

Example: Round Synchronization

How to share a secret so that it can be recovered by any 2 of 3 participants?



- Choose a random line, going through $(0, S)$.
- Give coordinates (i, S_i) to i^{th} participant.
- Any 2 participants can draw a line and check its value at 0.

What's next?

What's next?

- How to share secret for 2 of 10?

What's next?

- How to share secret for 2 of 10?
- How to share secret for 3 of 10?

What's next?

- How to share secret for 2 of 10?
- How to share secret for 3 of 10?
- How to check, if the dealer is not cheating?

What's next?

- How to share secret for 2 of 10?
- How to share secret for 3 of 10?
- How to check, if the dealer is not cheating?
- How to create a secret number so that not even the dealer would know it?

What's next?

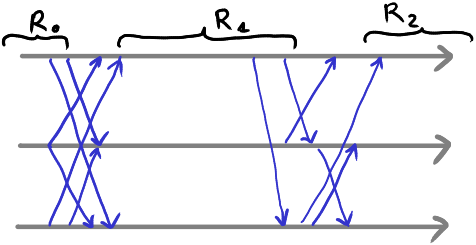
- How to share secret for 2 of 10?
- How to share secret for 3 of 10?
- How to check, if the dealer is not cheating?
- How to create a secret number so that not even the dealer would know it?

Distributed Key Generation – a class of algorithms doing just that.

- Often assume the synchronous setting.
  Time is split into rounds – non interleaving time periods.
- No message can include an information from the following rounds.
- But the internet is asynchronous!

Example: Round Synchronization

We have to simulate the sync rounds!



Will it work in all the cases?

?

- Each node sends a message upon completion of a round.
- Each node proceeds to the next round only upon receiving messages from all the nodes.
- Usually a byzantine setting is also assumed, making it more complicated.

Example: Round Synchronization

How to model it?

Considering it as an FSM

Example: Round Synchronization

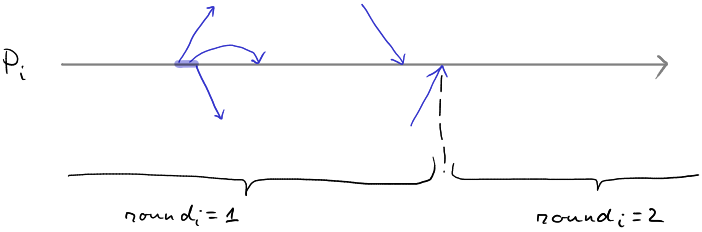
How to model it?

Considering it as an FSM

1. What is the state?

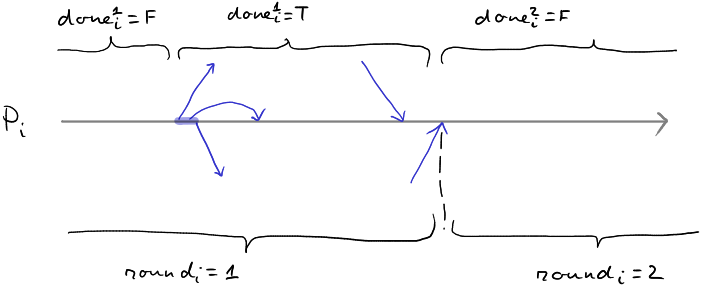
Example: Round Synchronization

What is the state?



Example: Round Synchronization

What is the state?



Example: Round Synchronization

What is the state?

$$round_n, done_n^r, \quad \forall n \in Nodes, r \in Rounds. \tag{9}$$

Example: Round Synchronization

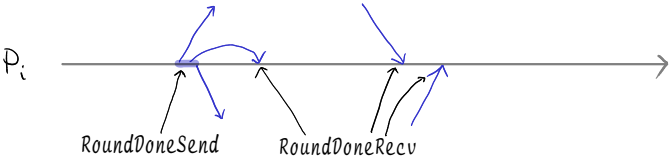
How to model it?

Considering it as an FSM

1. What is the state?
2. What are the events?

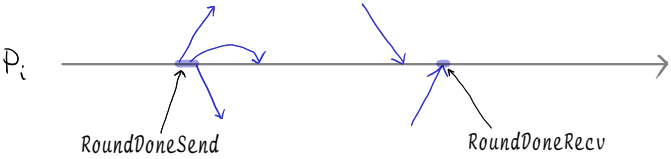
Example: Round Synchronization

What are the events?



Example: Round Synchronization

What are the events?



Example: Round Synchronization

What are the events?

$$RoundDoneSend_n, RoundDoneRecv_n, \quad \forall n \in Nodes. \tag{10}$$

Example: Round Synchronization

How to model it?

Considering it as an FSM

1. What is the state?
2. What are the events?
   Basic stuff for the FSM is done.
3. What properties we are going to check?

Example: Round Synchronization

What properties we are going to check?

- There is no two active rounds at the same time.

Example: Round Synchronization

What properties we are going to check?

- There is no two active rounds at the same time.
- A round r is considered active at node n if

$$round_n = r \wedge \neg done_n^r.$$

Example: Round Synchronization

What properties we are going to check?

- There is no two active rounds at the same time.
- A round r is considered active at node n if

$$round_n = r \wedge \neg done_n^r.$$

- Or – if two rounds are active, then they are the same.

Example: Round Synchronization

What properties we are going to check?

- There is no two active rounds at the same time.
- A round r is considered active at node n if

$$round_n = r \wedge \neg done_n^r.$$

- Or – if two rounds are active, then they are the same.
- More strictly:

$$\forall r_1, r_2 \in Rounds,\ n_1, n_2 \in Nodes :$$
$$round_{n_1} = r_1 \wedge \neg done_{n_1}^{r_1} \wedge$$
$$round_{n_2} = r_2 \wedge \neg done_{n_2}^{r_2}$$
$$\implies r_1 = r_2$$

Example: Round Synchronization

Live Coding!

After looking at functions, sequences, records
and a way to model messages.

Example: Round Synchronization

```
MODULE TLA_Lecture_Round
EXTENDS Naturals, TLAPS
CONSTANT NODES
CONSTANT ROUNDS
ASSUME NodesAssm ≜ ∃ R ∈ Nat : ROUNDS = 0 .. R
VARIABLE round
VARIABLE done
VARIABLE msgs
vars ≜ ⟨round, done, msgs⟩
TypeOK ≜
  ∧ round ∈ [NODES → ROUNDS]
  ∧ done  ∈ [NODES → [ROUNDS → BOOLEAN ]]
  ∧ msgs  ∈ SUBSET [r : ROUNDS, n : NODES]
```

Example: Round Synchronization

$$
\begin{array}{l}
\hline
RoundDoneSend(n) \triangleq \\
\quad \land \neg done[n][round[n]] \\
\quad \land done' = [done \text{ EXCEPT } ![n][round[n]] = \text{TRUE}] \\
\quad \land msgs' = msgs \cup \{[r \mapsto round[n],\ n \mapsto n]\} \\
\quad \land \text{UNCHANGED } round \\
\\
RoundDoneRecv(n) \triangleq \\
\quad \land round[n] + 1 \in ROUNDS \quad \text{For } TLC \text{ only.} \\
\quad \land \{[r \mapsto round[n],\ n \mapsto nn] : nn \in NODES\} \subseteq msgs \\
\quad \land round' = [round \text{ EXCEPT } ![n] = @ + 1] \\
\quad \land \text{UNCHANGED } \langle msgs,\ done \rangle
\end{array}
$$

Example: Round Synchronization

$$
\begin{aligned}
&\mathit{Init} \triangleq \\
&\quad \land\ \mathit{round} = [n \in \mathit{NODES} \mapsto 0] \\
&\quad \land\ \mathit{done} \ = [n \in \mathit{NODES} \mapsto [r \in \mathit{ROUNDS} \mapsto \text{FALSE}]] \\
&\quad \land\ \mathit{msgs} \ = \{\}
\end{aligned}
$$

$$
\begin{aligned}
&\mathit{Next} \triangleq \exists\, n \in \mathit{NODES} : \mathit{RoundDoneSend}(n) \lor \mathit{RoundDoneRecv}(n) \\
&\mathit{Live} \triangleq \mathrm{WF}_{\mathit{vars}}(\mathit{Next}) \\
&\mathit{Spec} \triangleq \mathit{Init} \land \Box[\mathit{Next}]_{\mathit{vars}} \land \mathit{Live}
\end{aligned}
$$

Example: Round Synchronization

$$
\begin{array}{l}
\hline
RoundsIsolated \triangleq \\
\quad \forall r1, r2 \in ROUNDS, n1, n2 \in NODES : \\
\quad\quad (\land r1 = round[n1] \land \neg done[n1][r1] \\
\quad\quad\ \ \land r2 = round[n2] \land \neg done[n2][r2] \\
\quad\quad) \Rightarrow r1 = r2 \\
\\
EachRoundIsReached \triangleq \\
\quad \forall n \in NODES, r \in ROUNDS : \Diamond(round[n] = r) \\
\\
\text{THEOREM } Spec \Rightarrow \\
\quad \land \Box TypeOK \\
\quad \land \Box RoundsIsolated \\
\quad \land EachRoundIsReached
\end{array}
$$

Outline

Structure of the TLA$^+$ Language

Example: Round Synchronization

Example: Connection Locks

Example: Brute-force Problems

TLA$^+$ Syntax in More Details

Common Modelling Tricks

Follow-up

References

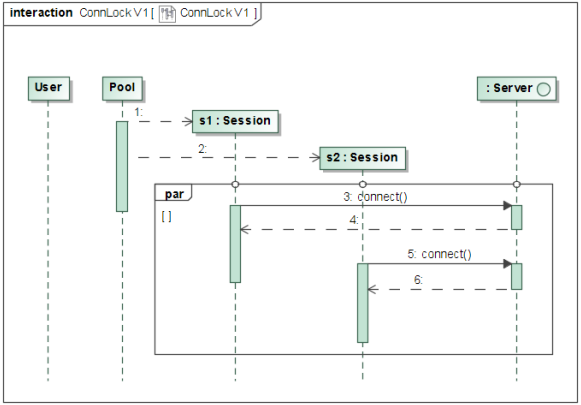
Connection Locks | Context

A fragment of a system in the telecommunication domain will be used as a running example for the introduction.

1. The problem with the system was encountered in the morning, a solution was found, although looked risky to make a deadlock.
2. TLA$^+$ specification was written and model checked ensuring absence of any deadlocks, then code was implemented. The solution was deployed till the noon (the entire solution took 3-4 hours).
3. The system was in production, so we had no time to test for absence of deadlocks using the typical testing methods (it takes long time to do that).

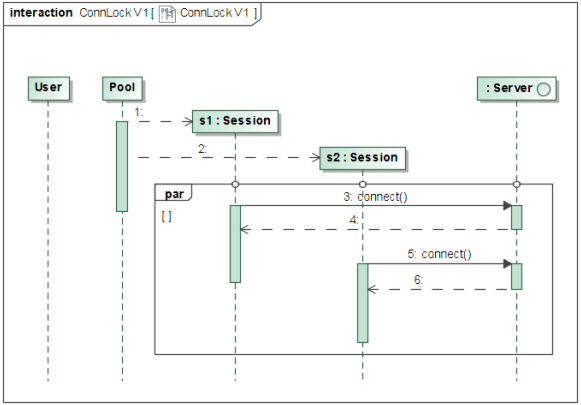
Connection Locks | Initial Design

1. Connections (≈ 60/node) are maintained in a pool.
2. All the connections should be initialized on start-up to serve requests fast later on.



Connection Locks | Initial Design

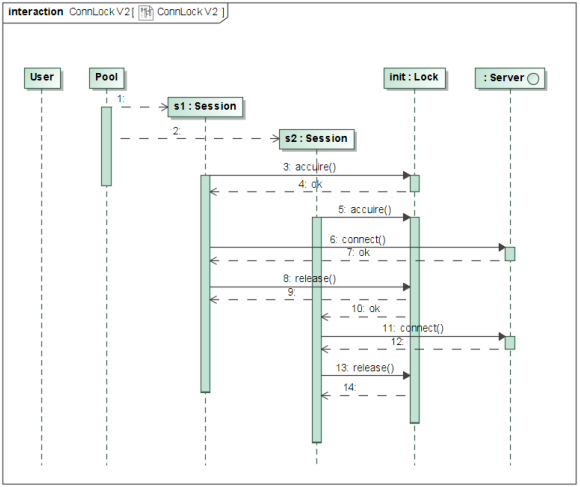
1. Connections (≈ 60/node) are maintained in a pool.
2. All the connections should be initialized on start-up to serve requests fast later on.
3. **PROBLEM:** If all the connections are initialized concurrently, the target system crashes due to its limitations.



Connection Locks | Initialization Lock

First attempt to solve the problem:

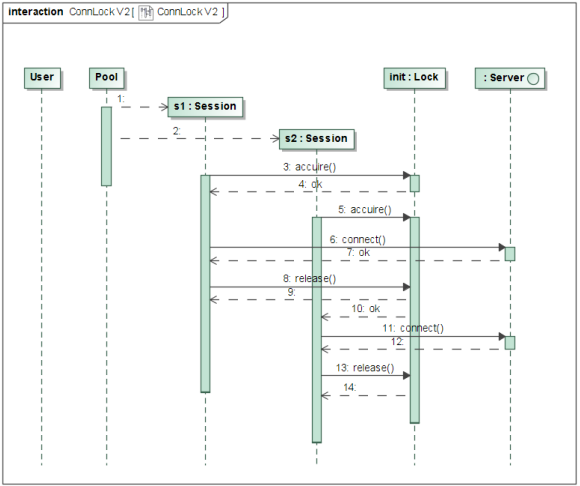
1. Use a lock to serialize all the initializations.
2. Initialization of a connection takes long time (3-5s).
3. It takes $\approx$ 4 minutes to initialize 60 connections.



Connection Locks | Initialization Lock

First attempt to solve the problem:

1. Use a lock to serialize all the initializations.
2. Initialization of a connection takes long time (3-5s).
3. It takes ≈ 4 minutes to initialize 60 connections.



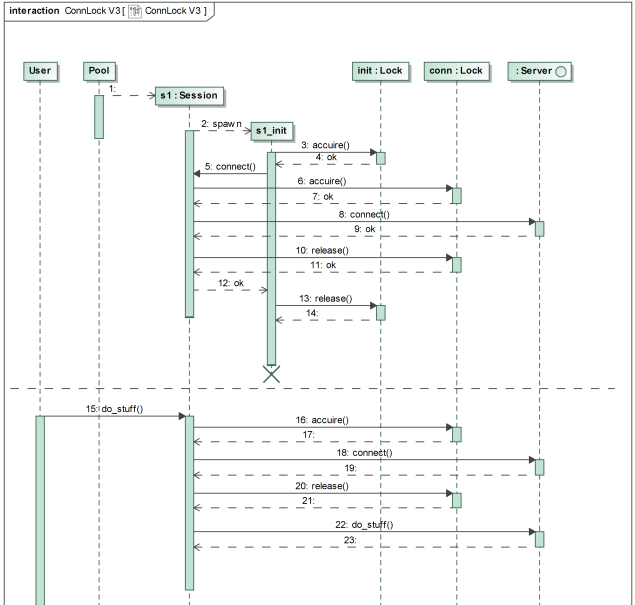
PROBLEM: The system is blocked for 4 minutes after the start-up.

Connection Locks | Connection Lock Added

To solve the blocking of the system on start-up without allowing parallel connections:

1. A second lock was introduced:
   1.1 conn:Lock for serializing all the attempts to make new connection.
   1.2 init:Lock to serialize all the connection start-up time initializations.
2. On a service request (call), only connLock is acquired if connection is not established yet.
3. On the initialization, initLock and then connLock is acquired for each connection to make the actual operation.

Connection Locks | Connection Lock Added



Connection Locks | Structure of the State

```
-------- MODULE TLA_Lecture_ConnLock_v1 -------
CONSTANT S                \* A set of sessions.
VARIABLE connLock         \* The connection lock.
VARIABLE initLock         \* The initialization lock.
VARIABLE initProc         \* State of the initialization process.
VARIABLE callProc         \* State of the process calling the session.
VARIABLE sessions         \* State of the session (connected or not).
vars == <<connLock, initLock, initProc, callProc, sessions>>
Free == CHOOSE s : s \notin S
...
=============================================
```

The variables are mapped with the presented diagrams as follows:

- $initProc[s]$ and $sessions[s]$ stand for the combined state of the `s:Session` and `s:Init` processes, in the context of the initialization.
- $callProc[s]$ and $sessions[s]$ stand for the state of the `s:Session` only, in the context of a call.

The model can be instantiated using:

$$S \leftarrow \{s_1, s_2, s_3, s_4\}. \tag{11}$$

Connection Locks | Initialization and the Spec

```
Init == /\ connLock = Free
        /\ initLock = Free
        /\ initProc = [s \in S |-> "wait"]
        /\ callProc = [s \in S |-> "wait"]
        /\ sessions = [s \in S |-> FALSE]
Next == \E s \in S : OnInit(s) \/ OnCall(s)
Live == WF_vars(Next)
Spec == Init /\ [][Next]_vars /\ Live
```

Initially:

- Both locks are free (not locked);
- All the processes are waiting for their turn;
- All the sessions are not connected.

The liveness properties are needed to eliminate executions, where nothing happens. They are only needed when checking the temporal properties.

Connection Locks | Properties

```
TypeOK == /\ initLock \in S \cup {Free}
          /\ connLock \in S \cup {Free}
          /\ initProc \in [S -> {"wait", "init", "conn", "done"}]
          /\ callProc \in [S -> {"wait",         "conn", "done"}]
          /\ sessions \in [S -> BOOLEAN]

SessionWillBeConnected ==
  <>[]\A s \in S :
      /\ sessions[s]
      /\ callProc[s] = "done"
      /\ initProc[s] = "done"
      /\ initLock = Free
      /\ connLock = Free

THEOREM Spec => /\ []TypeOK
                /\ SessionWillBeConnected
```

Here

- *TypeOK* is an invariant asserting type correctness of the state.
- *SessionWillBeConnected* is a temporal property.
- When using the TLC, the theorem is only a hint for a user on what should be checked.

Connection Locks | Actions (OnInit)

The following actions describe the initialization procedure:

```
OnInitLock(s) == \* Asynchronous initialization: init lock acquired
  /\ initProc[s] = "wait"
  /\ initLock = Free
  /\ initLock' = s
  /\ initProc' = [initProc EXCEPT ![s] = "init"]
  /\ UNCHANGED <<connLock, callProc, sessions>>

OnInitConn(s) == \* Asynchronous initialization: connect lock acquired
  /\ initProc[s] = "init"
  /\ Connected(s) \/ ConnectStarted(s)
  /\ initProc' = [initProc EXCEPT ![s] = "conn"]
  /\ UNCHANGED <<initLock, callProc>>

OnInitDone(s) == \* Asynchronous initialization: connect done, locks released
  /\ initProc[s] = "conn"
  /\ Connected(s) \/ ConnectSuccess(s)
  /\ initProc' = [initProc EXCEPT ![s] = "done"]
  /\ initLock' = Free
  /\ UNCHANGED <<callProc>>

OnInit(s) == OnInitLock(s) \/ OnInitConn(s) \/ OnInitDone(s)
```

Actions can be considered as combinations of pre-conditions and post-conditions.

Connection Locks | Actions (OnCall)

Actions modelling an explicit call (maybe during the initialization process):

```
OnCallConn(s) == \* Connect on explicit call: connect lock acquired.
  /\ callProc[s] = "wait"
  /\ ConnectStarted(s)
  /\ callProc' = [callProc EXCEPT ![s] = "conn"]
  /\ UNCHANGED <<initLock, initProc>>

OnCallDone(s) == \* Connect on explicit call: connect done, lock released
  /\ callProc[s] = "conn"
  /\ ConnectSuccess(s)
  /\ callProc' = [callProc EXCEPT ![s] = "done"]
  /\ UNCHANGED <<initLock, initProc>>

OnCallConnected(s) == \* Connected, just serve the requests.
  /\ callProc[s] \in {"wait", "done"}
  /\ Connected(s)
  /\ callProc' = [callProc EXCEPT ![s] = "done"]
  /\ UNCHANGED <<initLock, initProc>>

OnCall(s) == OnCallConn(s) \/ OnCallDone(s) \/ OnCallConnected(s)
```

Connection Locks | Other Formulas (Connection)

The following formulas are used in several places in the actions shown above. They are responsible for the *connLock* and *sessions* variables.

```
Connected(s) == sessions[s] /\ UNCHANGED <<connLock, sessions>>

ConnectStarted(s) ==
  /\ ~sessions[s]
  /\ connLock = Free
  /\ connLock' = s
  /\ UNCHANGED <<sessions>>

ConnectSuccess(s) ==
  /\ ~sessions[s]
  /\ connLock = s
  /\ sessions' = [sessions EXCEPT ![s] = TRUE]
  /\ connLock' = Free
```

Connection Locks | Complete Spec and Demo

The main points:

- Development of the entire fix including code as well as formal specification and model checking took 3-4 hours.
- Reduced risk of the fix and the time needed for testing.
- $\implies$ Formal methods can be used to solve problems quickly.

The complete specification: spec/TLA_Lecture_ConnLock_v1.pdf

Outline

Structure of the TLA$^+$ Language

Example: Round Synchronization

Example: Connection Locks

Example: Brute-force Problems

TLA$^+$ Syntax in More Details

Common Modelling Tricks

Follow-up

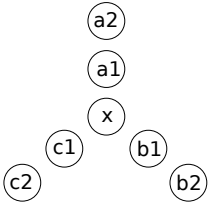
References

Example: Brute-force Problems

A problem from a math. Olympiad for a 3rd class :)

There are 7 distinct numbers from 1 to 7. They are written in a star with three rays. One number is in the middle and other are on the rays. A sum of numbers on each ray is 12. How can the numbers be arranged? [See also ▸ DieHard.tla]

```
VARIABLE x, a1, a2, b1, b2, c1, c2
TypeOK ≜
    ∧  x ∈ 1 . . 7 ∧ a1 ∈ 1 . . 7 ∧ a2 ∈ 1 . . 7
                   ∧ b1 ∈ 1 . . 7 ∧ b2 ∈ 1 . . 7
                   ∧ c1 ∈ 1 . . 7 ∧ c2 ∈ 1 . . 7
    ∧  {x, a1, a2, b1, b2, c1, c2} = 1 . . 7
    ∧  x + a1 + a2 = 12
    ∧  x + b1 + b2 = 12
    ∧  x + c1 + c2 = 12
vars ≜ ⟨x, a1, a2, b1, b2, c1, c2⟩
Init ≜ TypeOK
Next ≜ UNCHANGED vars
Spec ≜ Init ∧ □[Next]_vars
THEOREM Spec ⇒ ¬TypeOK   Just to get an answer
```



▸ specs/TLA_Lecture_Olymp_3C_1to7_v1.pdf

Outline

Structure of the TLA$^+$ Language

Example: Round Synchronization

Example: Connection Locks

Example: Brute-force Problems

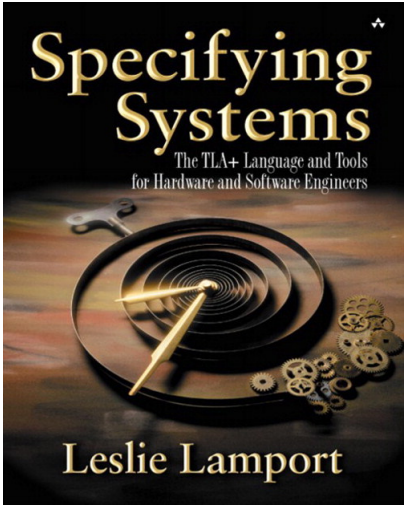
TLA$^+$ Syntax in More Details

Common Modelling Tricks

Follow-up

References

TLA$^+$ | Recommended Sources



Leslie Lamport. Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers, 2002, Addison-Wesley Longman Publishing Co., Inc., Boston, MA, USA, ISBN: 032114306X

See also:

- TLA+ Video Course
- https://lamport.azurewebsites.net/tla/tla.html

TLA$^+$ | Formulas

In the TLA$^+$ language there are several kinds of formulas:

TLA$^+$ | Formulas

In the TLA$^+$ language there are several kinds of formulas:

1. State functions, predicates (boolean valued functions). They contain only unprimed variables. E.g.:

$$a + b, \quad a = b, \quad \exists c \in Nat : a = b * c. \tag{12}$$

If considered as temporal formulas, they apply for the first step of the execution.

TLA$^+$ | Formulas

In the TLA$^+$ language there are several kinds of formulas:

1. State functions, predicates (boolean valued functions). They contain only unprimed variables. E.g.:

$$a + b, \quad a = b, \quad \exists c \in Nat : a = b * c. \tag{12}$$

   If considered as temporal formulas, they apply for the first step of the execution.

2. Actions – formulas containing primed variables, e.g.:

$$n' = n + 1, \quad n = n' - 1. \tag{13}$$

   As temporal formulas, they apply for the first two steps of the execution.

TLA$^+$ | Formulas

In the TLA$^+$ language there are several kinds of formulas:

1. State functions, predicates (boolean valued functions). They contain only unprimed variables. E.g.:

$$a + b, \quad a = b, \quad \exists c \in Nat : a = b * c. \tag{12}$$

   If considered as temporal formulas, they apply for the first step of the execution.

2. Actions – formulas containing primed variables, e.g.:

$$n' = n + 1, \quad n = n' - 1. \tag{13}$$

   As temporal formulas, they apply for the first two steps of the execution.

3. Temporal formulas assign boolean values to the behaviours. The must be invariant to stuttering.

$$Valid \triangleq \Box[n' = n + 1]_n, \quad Invalid \triangleq \Box(n' = n + 1). \tag{14}$$

   A stuttering step over variables in *vars* is such a step, where all of the variables remain unchanged.

TLA$^+$ | Operators, Logic

$a \wedge b$	`a /\ b`
$a \vee b$	`a \/ b`
$\neg a$	`~a`
$a \Rightarrow b$	`a => b`
$a \equiv b$	`a <=> b`
$\forall a : p$	`\A a : p`
$\exists a : p$	`\E a : p`
$\forall a \in A : p$	`\A a \in A : p`
$\exists a \in A : p$	`\E a \in A : p`
$\text{CHOOSE } a : p$	`CHOOSE a : p`
$\text{CHOOSE } a \in A : p$	`CHOOSE a \in A : p`

$\text{BOOLEAN} = \{\text{TRUE}, \text{FALSE}\}$

TLA$^+$ | Operators, Logic

$a \wedge b$	`a /\ b`
$a \vee b$	`a \/ b`
$\neg a$	`~a`
$a \Rightarrow b$	`a => b`
$a \equiv b$	`a <=> b`
$\forall a : p$	`\A a : p`
$\exists a : p$	`\E a : p`
$\forall a \in A : p$	`\A a \in A : p`
$\exists a \in A : p$	`\E a \in A : p`
$\text{CHOOSE } a : p$	`CHOOSE a : p`
$\text{CHOOSE } a \in A : p$	`CHOOSE a \in A : p`

$\text{BOOLEAN} = \{\text{TRUE}, \text{FALSE}\}$

- Vertical alignment of $\wedge$ and $\vee$ can be used instead of ().
- Quantifiers can be grouped, e.g. $\forall a, b \in S, m \in M : p$.
- CHOOSE stands for Hilbert's ε. It is deterministic.

TLA$^+$ | Operators, Logic

$a \wedge b$	`a /\ b`
$a \vee b$	`a \/ b`
$\neg a$	`~a`
$a \Rightarrow b$	`a => b`
$a \equiv b$	`a <=> b`
$\forall a : p$	`\A a : p`
$\exists a : p$	`\E a : p`
$\forall a \in A : p$	`\A a \in A : p`
$\exists a \in A : p$	`\E a \in A : p`
$\text{CHOOSE } a : p$	`CHOOSE a : p`
$\text{CHOOSE } a \in A : p$	`CHOOSE a \in A : p`

$\text{BOOLEAN} = \{\text{TRUE}, \text{FALSE}\}$

- Vertical alignment of $\wedge$ and $\vee$ can be used instead of ().
- Quantifiers can be grouped, e.g. $\forall a, b \in S, m \in M : p$.
- CHOOSE stands for Hilbert's ε. It is deterministic.

Examples for CHOOSE:

$a/b \triangleq \text{CHOOSE } c \in Real : a = b * c,$

$Free \triangleq \text{CHOOSE } s : s \notin S,$

$AnyNat \triangleq \text{CHOOSE } x \in Nat : \text{TRUE}.$

TLA$^+$ | Operators, Sets

$A = B$	`A = B`
$A \neq B$	`A # B`
$a \in A$	`a \in A`
$a \notin A$	`a \notin A`
$A \cup B$	`A \cup B`
$A \cap B$	`A \cap B`
$A \subseteq B$	`A \subseteq B`
$A \setminus B$	`A \ B`

TLA$^+$ | Operators, Sets

$A = B$	`A = B`
$A \neq B$	`A # B`
$a \in A$	`a \in A`
$a \notin A$	`a \notin A`
$A \cup B$	`A \cup B`
$A \cap B$	`A \cap B`
$A \subseteq B$	`A \subseteq B`
$A \setminus B$	`A \ B`

$\{e_1, e_2, e_3\}$ – literal.
$\{x \in S : p(x)\}$ – filtering.
$\{e(x) : x \in S\}$ – mapping.

SUBSET S – powerset of S.
UNION S – union of sets in S.

TLA$^+$ | Operators, Sets

$A = B$	`A = B`
$A \neq B$	`A # B`
$a \in A$	`a \in A`
$a \notin A$	`a \notin A`
$A \cup B$	`A \cup B`
$A \cap B$	`A \cap B`
$A \subseteq B$	`A \subseteq B`
$A \setminus B$	`A \ B`

$\{e_1, e_2, e_3\}$ – literal.
$\{x \in S : p(x)\}$ – filtering.
$\{e(x) : x \in S\}$ – mapping.

$\text{SUBSET } S$ – powerset of S.
$\text{UNION } S$ – union of sets in S.

Note: $a \subseteq A$ vs $a \in \text{SUBSET } A$.
Add element to a set: $A \cup \{a\}$.
Remove element from a set: $A \setminus \{a\}$.

TLA$^+$ | Functions (1/2)

Functions in TLA$^+$ are considered as mathematical constructs, i.e. relations (instead of functions in the sense of programming languages).

- $F \triangleq [D \to R]$ – a set of *all* functions with domain D and range R.
- $f \triangleq [x \in D \mapsto e(x)]$ – a function with domain D and values $e(x)$. Note that $\{e(x) : x \in D\} \subseteq R \Rightarrow f \in F$.

$D = \{a, b\}$ $\quad R = \{x, y, z\}$

$$
\begin{aligned}
[D \to R] = \{ & \\
& \{(a, x), (b, x)\}, \quad \{(a, y), (b, x)\}, \quad \{(a, z), (b, x)\}, \\
& \{(a, x), (b, y)\}, \quad \{(a, y), (b, y)\}, \quad \{(a, z), (b, y)\}, \\
& \{(a, x), (b, z)\}, \quad \{(a, y), (b, z)\}, \quad \{(a, z), (b, z)\} \}
\end{aligned}
$$

TLA$^+$ | Functions (2/2)

Operators for working with functions:

TLA$^+$ | Functions (2/2)

Operators for working with functions:

- $D = \text{DOMAIN } f$ – returns a domain of a function.

TLA$^+$ | Functions (2/2)

Operators for working with functions:

- $D = \text{DOMAIN } f$ – returns a domain of a function.
- $[f \text{ EXCEPT } ![x_1] = y_2]$ – a function similar to f, except that $f[x_1] = y_2$. The domain of the function cannot be changed in this way.

TLA$^+$ | Functions (2/2)

Operators for working with functions:

- $D = \text{DOMAIN } f$ – returns a domain of a function.
- $[f \text{ EXCEPT } ![x_1] = y_2]$ – a function similar to f, except that $f[x_1] = y_2$. The domain of the function cannot be changed in this way.
- To extend the domain of f by $17 \mapsto 13$, use $[x \in \text{DOMAIN } f \cup \{17\} \mapsto \text{IF } x \in \text{DOMAIN } f \text{ THEN } f[x] \text{ ELSE } 13]$.

TLA$^+$ | Functions (2/2)

Operators for working with functions:

- $D = \text{DOMAIN } f$ – returns a domain of a function.
- $[f \text{ EXCEPT } ![x_1] = y_2]$ – a function similar to f, except that $f[x_1] = y_2$. The domain of the function cannot be changed in this way.
- To extend the domain of f by $17 \mapsto 13$, use $[x \in \text{DOMAIN } f \cup \{17\} \mapsto \text{IF } x \in \text{DOMAIN } f \text{ THEN } f[x] \text{ ELSE } 13]$.
- $f[x_1]$ – application of a function (image of x_1 under f).
- @ refers to the existing value, e.g. $f' = [f \text{ EXCEPT } ![x_1] = @ \cup \{z_1\}]$.

TLA$^+$ | Functions (2/2)

Operators for working with functions:

- $D = \text{DOMAIN } f$ – returns a domain of a function.
- $[f \text{ EXCEPT } ![x_1] = y_2]$ – a function similar to f, except that $f[x_1] = y_2$. The domain of the function cannot be changed in this way.
- To extend the domain of f by $17 \mapsto 13$, use $[x \in \text{DOMAIN } f \cup \{17\} \mapsto \text{IF } x \in \text{DOMAIN } f \text{ THEN } f[x] \text{ ELSE } 13]$.
- $f[x_1]$ – application of a function (image of x_1 under f).
- @ refers to the existing value, e.g. $f' = [f \text{ EXCEPT } ![x_1] = @ \cup \{z_1\}]$.
- $f[x][y]$ – a function of two parameters (function of functions).

TLA$^+$ | Recursive Functions

Recursive functions can be defined as follows:

$$fact[n \in Nat] \triangleq \text{IF } n = 0 \text{ THEN } 1 \text{ ELSE } n * fact[n-1]. \tag{15}$$

It is an abbreviation of:

$$\begin{aligned} fact \triangleq \text{CHOOSE } fact : fact = [n \in Nat : \\ \text{IF } n = 0 \text{ THEN } 1 \\ \text{ELSE } n * fact[n-1]] \end{aligned} \tag{16}$$

Recursive functions are used rarely. They work with TLC, but are not supported by the TLAPS (yet). In most cases the set operations and TLA$^+$ actions are enough (looks opposite to functional languages).

TLA$^+$ functions can be considered often as maps / hash-tables.

TLA$^+$ | Functions vs Operators

Operators are definitions with parameters, e.g.

$$Max(x, y) \triangleq \text{IF } x > y \text{ THEN } x \text{ ELSE } y \tag{17}$$

While function in TLA$^+$ is a value, the operator is not. It looks more like a function in a programming languages.

Operators can also be defined as recursive:

$$\begin{aligned} &\text{RECURSIVE } fact(_) \\ &fact(n) \triangleq \text{IF } n = 0 \text{ THEN } 1 \text{ ELSE } n * fact(n-1) \end{aligned} \tag{18}$$

TLA$^+$ | Records (1/2)

Records can be used to represent composite data structures:

TLA$^+$ | Records (1/2)

Records can be used to represent composite data structures:

- $R \triangleq [name : Names, age : Nat]$ — a set of all records with fields *name* and *age*.

TLA$^+$ | Records (1/2)

Records can be used to represent composite data structures:

- $R \triangleq [name : Names, age : Nat]$ — a set of all records with fields *name* and *age*.
- $r \triangleq [name \mapsto "Jonas", age \mapsto 32]$ — a particular record instance, $r \in R$.

TLA$^+$ | Records (1/2)

Records can be used to represent composite data structures:

- $R \triangleq [name : Names, age : Nat]$ — a set of all records with fields *name* and *age*.
- $r \triangleq [name \mapsto "Jonas", age \mapsto 32]$ — a particular record instance, $r \in R$.
- $older = [r \text{ EXCEPT } !.age = @ + 1]$ — records can be updated similarly to the functions.

TLA$^+$ | Records (1/2)

Records can be used to represent composite data structures:

- $R \triangleq [name : Names, age : Nat]$ — a set of all records with fields *name* and *age*.
- $r \triangleq [name \mapsto "Jonas", age \mapsto 32]$ — a particular record instance, $r \in R$.
- $older = [r \text{ EXCEPT } !.age = @ + 1]$ — records can be updated similarly to the functions.
- $older.age = 33$ — fields can be accessed using the dot notation. The same is for nested records.

TLA$^+$ | Records (2/2)

The records are functions with $\text{DOMAIN } r = \{"name", "age"\}$. They can be manipulated as functions, e.g. $older["age"] = 33$.

TLA$^+$ | Records (2/2)

The records are functions with $\text{DOMAIN } r = \{"name", "age"\}$. They can be manipulated as functions, e.g. $older["age"] = 33$.

Records are used to model:

- state of the processes. Two alternatives:

$$name \in [Person \to Names] \land age \in [Person \to Age]; \tag{19}$$

$$person \in [Person \to [name : Names, age : Age]]. \tag{20}$$

- messages sent between processes.
- ...

TLA$^+$ | Tuples and Sequences

A tuple is a way to represent an ordered pair, triplet, etc., e.g. $\langle \boldsymbol{a}, \boldsymbol{b}, \boldsymbol{c} \rangle$ is written as `<<a, b, c>>`. A set of all tuples of a particular structure is defined by the Cartesian product:

$$t \triangleq \langle \text{"}jonas\text{"}, 33 \rangle; \quad t \in Names \times Nat. \tag{21}$$

Tuples are functions with domains $D \in \mathrm{SUBSET}\ Nat$, i.e.

$$t = [1 \mapsto \text{"}jonas\text{"}, 2 \mapsto 33]. \tag{22}$$

Particular elements of the tuple can be accessed as $t[2] = 33$.

There is a standard module ***Sequences***, providing functions like:

- $Seq(S)$ — a set of all finite sequences with elements in S.
- $Len(s)$ — length of the sequence s.
- $Append(s, e)$ — append element e to the sequence s, etc.

TLA$^+$ | Conditional and Structural Elements

There are two kinds of conditional statements in TLA$^+$. Both of them are expressions, thus have a value. All of the following constructions can be expressed using disjunctions.

$$
\begin{aligned}
v_1 &= \text{IF } cond \text{ THEN } exp_a \text{ ELSE } exp_b \\
v_2 &= \text{CASE } cond_1 \rightarrow exp_1 \ \Box \ cond_2 \rightarrow exp_2 \ \Box \ cond_3 \rightarrow exp_3 \\
v_3 &= \text{CASE } cond_1 \rightarrow exp_1 \ \Box \ \text{OTHER} \rightarrow exp_n
\end{aligned} \tag{23}
$$

There is a LET construction to allow local definitions in the expressions:

$$
\text{LET } d_1 \triangleq exp_1 \ d_2 \triangleq exp_2 \text{ IN } exp(d_1, d_2) \tag{24}
$$

LET is also an expression, its value equals to $exp(exp_1, exp_2)$.

TLA$^+$ | Action Operators

All the temporal formulas in TLA$^+$ must be invariant under stuttering. Next two operators are introduced to handle that easier.

- Consider $\Box[Next]_{vars}$. This means that always either *Next* or stuttering step will occur, i.e.

$$[A]_v \equiv A \vee (v' = v) \tag{25}$$

- Sometimes we want to say, that some action will occur, in that case we can say $\Diamond\langle Connected\rangle_v$ where

$$\langle A\rangle_v \equiv A \wedge (v' \neq v) \tag{26}$$

Other operators:

$$\text{ENABLED } A \equiv \text{Action } A \text{ can be performed at this state.} \tag{27}$$

$$\text{UNCHANGED } v \equiv v' = v \tag{28}$$

TLA$^+$ | Temporal Logic

There are several important temporal formulas:

- $\Box F$ — F is always true (in this and each following step of an execution).
- $\Diamond F \equiv \neg\Box\neg F$ — F is eventually true.
- $F \rightsquigarrow G \equiv \Box(F \Rightarrow \Diamond G)$ — F leads to G.

  I.e. if F became true, then G will become true eventually.
- $\Box\Diamond F$ — F will be true infinitely many times.
- $\Diamond\Box F$ — when F will become true, it will remain so.

Fairness can be specified using:

- $\mathrm{WF}_v(A) \equiv \Box(\Box\mathrm{ENABLED}\ \langle A\rangle_v \Rightarrow \Diamond\langle A\rangle_v)$ — an action A changing variables in v will eventually occur, if it will be enabled infinitely.
- $\mathrm{SF}_v(A) \equiv \Diamond\Box(\neg\mathrm{ENABLED}\ \langle A\rangle_v) \vee \Box\Diamond\langle A\rangle_v$ — the action A is either disabled forever, or it will occur infinitely many times.

Outline

Structure of the TLA$^+$ Language

Example: Round Synchronization

Example: Connection Locks

Example: Brute-force Problems

TLA$^+$ Syntax in More Details

Common Modelling Tricks

Follow-up

References

Use Sets Instead of Counts for Constants

It is usually better to use sets as constant values instead of set cardinalities.

CONSTANT *Nodes*
ASSUME $\exists n \in Nodes : \text{TRUE}$

CONSTANT *NodeCount*
ASSUME $NodeCount \geq 1$
$Nodes \triangleq [1..NodeCount]$

In this way:

- We leave node identities abstract (as opposite to naturals).
- We can use model values and symmetry reduction in TLC.

NOTE: The same usually applies to quorums. A set of quorums allows more possibilities than quorums based in their size. E.g. we have a special node that is important ant thus is online, thus we can have smaller quorums, just they need to include that special node.

Modelling a distributed system

- All the actions must only refer to variables in a single node.
  - Only messages can pass information between nodes.
  - A node can only make decisions on its local information.
  - An action can change state of a single node only.
- Checked properties can use variables of different nodes.
- Use $[Node \to State]$ for modelling uniform nodes.

```
MODULE TLA_Lecture_Tips_DistSys
VARIABLES client, server, msgs
vars ≜ ⟨client, server, msgs⟩
Msgs ≜ [t : {"event"}]
EventSend ≜ ∧ client = "pending"
            ∧ client′ = "sent"
            ∧ msgs′ = msgs ∪ {[t ↦ "event"]}
            ∧ UNCHANGED server
EventRecv ≜ ∃ m ∈ msgs : ∧ m.t = "event"
                         ∧ server = "waiting"
                         ∧ server′ = "processing"
                         ∧ UNCHANGED ⟨client, msgs⟩
Next ≜ EventSend ∨ EventRecv
Init ≜ ∧ client = "pending"
       ∧ server = "waiting"
       ∧ msgs = {}
Spec ≜ Init ∧ □[Next]_vars
RecvImpliesSent ≜ server = "processing" ⇒ client = "sent"
THEOREM Spec ⇒ □RecvImpliesSent
```

Modelling a Network

A backboard approach

The network is modelled by:

- A $msgs \subseteq Msgs$ variable for all messages that were sent.
- Use $msgs' = msgs \cup sentMsgs$ for modelling message sending.
- Use $\exists m \in msgs : \ldots$ for modelling reception of messages.

In this way:

- Duplicated, dropped, reordered messages are modelled "for free".
- There is no need to remove message from the *msgs* on reception.
- Using $Seq(Msgs)$ increases state space, and often not needed.
- *WF* and *SF* can make this approach to not model failed deliveries.
- NOTE: It is hard to ensure FIFO, even with TCP (reconnects).
- NOTE: See previous slide for an example.

Modelling a Message Accumulation (LiveCoding!)

Quite often an algorithm has to collect a set of messages.

- E.g. from a quorum of nodes.
- We had an example in the Synchronous Round specification.

Try to receive them at once.

- That will reduce the state space.
- Only if the collected state doesn't affect other parts of the algorithm.

```
RoundDoneRecv(n) ==
    /\ round[n] + 1 \in ROUNDS \* For TLC only.
    /\ {[r |-> round[n], n |-> nn] : nn \in NODES} \subseteq msgs
    /\ round' = [round EXCEPT ![n] = @+1]
    /\ UNCHANGED <<msgs, done>>
```

instead of

```
RoundDoneRecv(n) ==
    /\ round[n] + 1 \in ROUNDS \* For TLC only.
    /\ \E m \in msgs :
         /\ m.r = round[n]
         /\ recv' = [recv EXCEPT ![n][m.r] = @ \cup {m.n}]
         /\ \/ /\ Cardinality(recv'[n][m.r]) = Cardinality(NODES)
               /\ round' = [round EXCEPT ![n] = @+1]
               /\ UNCHANGED <<msgs, done>>
            \/ /\ Cardinality(recv'[n][m.r]) < Cardinality(NODES)
               /\ UNCHANGED <<msgs, round, done>>
```

Modelling a Network

Network partitions

The partitioning can be modelled by:

- Assuming CONSTANT *Nodes*.
- Introduce VARIABLE *net*.
- With type $net \in [Nodes \rightarrow [Nodes \rightarrow BOOLEAN]]$.
- Where $net[src][dst]$ means messages can be sent from *src* to *dst*.
- Change receiving to $\exists m \in msgs : net[m.from][node] \wedge \ldots$
- Use *Init* to define network topology and possible partitions.
  - Any partial partitions, including asymmetric (choose vs exists).
  - Symmetric partitions only (e.g. Alfatafta et al. (2020)).
  - Complete partitions only.
  - Any other assumptions, e.g. $\exists$ interconnected quorum.

In this way:

- Fairness conditions do not make all the messages "eventually delivered".

NOTE: See a spec in the next slide.

Modelling a Network (cont.)

Network partitions

```
MODULE TLA_Lecture_Tips_Partitions
CONSTANTS Nodes, Quorums, Byzantine
VARIABLE net
TypeOK ≜
  ∧ net ∈ [Nodes → [Nodes → BOOLEAN ]]   net[src][dst]
NetAnyPartialPartitionsExist ≜
  ∧ ∃ q ∈ Quorums :
      ∧ q ∩ Byzantine = {}              Exists not-byzantine quorum
      ∧ ∀ ns, nd ∈ q : net[ns][nd]      and it is inter-connected.
  ∧ ∀ n ∈ Nodes : net[n][n]             Nodes can access themselves.
NetAnyPartialPartitionsChoose ≜
  LET q ≜ CHOOSE q ∈ Quorums : TRUE     NOTE: To decrease number of initial states
  IN    ∧ q ∩ Byzantine = {}            Exists not-byzantine quorum
        ∧ ∀ ns, nd ∈ q : net[ns][nd]    and it is inter-connected.
        ∧ ∀ n ∈ Nodes : net[n][n]       Nodes can access themselves.
NetAnyPartialPartitions ≜ NetAnyPartialPartitionsChoose
NetSymmetricPartialPartitions ≜
  ∧ NetAnyPartialPartitions
  ∧ ∀ n1, n2 ∈ Nodes : net[n1][n2] = net[n2][n1]   No one-way connections (and adversaries).
NetCompletePartitions ≜
  ∧ NetSymmetricPartialPartitions
  ∧ ∀ n1, n2 ∈ Nodes : ¬net[n1][n2] ⇒ ¬∃ n3 ∈ Nodes : net[n1][n3] ∧ net[n3][n2]
Init ≜ net ∈ [Nodes → [Nodes → BOOLEAN ]] ∧ NetAnyPartialPartitions
```

Testcases for a spec

Sometimes it takes long time to find a counterexample, we can use the state found as a testcase to check the same situation faster.

```
TestInit1 ==
  /\ a = <<{}, {2, 3, 4}, {1, 3, 4}, {1, 3, 4}>>
  /\ b = <<{}, {2, 3, 4}, {3, 4}, {3, 4}>>
  /\ c = <<{}, {2, 3, 4}, {2, 3, 4}, {2, 3, 4}>>
  /\ net = << <<TRUE, FALSE, TRUE, TRUE>>,
              <<FALSE, TRUE, TRUE, TRUE>>,
              <<FALSE, TRUE, TRUE, TRUE>>,
              <<FALSE, TRUE, TRUE, TRUE>> >>
TestInit2 ==
  /\ a = <<{}, {   2, 3, 4}, {   2, 3, 4}, {1, 2,    4}>>
  /\ b = <<{}, {   2, 3   }, {   2, 3   }, {   2, 3, 4}>>
  /\ c = <<{}, {   2, 3, 4}, {   2, 3, 4}, {   2, 3, 4}>>
  /\ net = << <<TRUE,  FALSE, FALSE, TRUE>>,
              <<FALSE, TRUE,  TRUE,  TRUE>>,
              <<FALSE, TRUE,  TRUE,  TRUE>>,
              <<FALSE, TRUE,  TRUE,  TRUE>> >>
TestInit == TestInit1 \/ TestInit2
TestNext == UNCHANGED vars
TestSpec == TestInit /\ [][TestNext]_vars
TestCheck ==
  \/ \E q \in Quorums, d \in SUBSET Nodes:
      /\ d # {}
      /\ \A qn \in q: c[qn] # {} /\ Decision(qn) = d
  \/ Print(<<Decision(1), Decision(2), Decision(3), Decision(4)>>, FALSE)
```

NOTE: *TestInitX* – particular test cases, *TestCheck* – checks the invariant in question or prints relevant information otherwise.

Outline

Structure of the TLA$^+$ Language

Example: Round Synchronization

Example: Connection Locks

Example: Brute-force Problems

TLA$^+$ Syntax in More Details

Common Modelling Tricks

Follow-up

References

Research

Main conference:

- TLA$^+$ Community Event, `https://conf.tlapl.us/home/`

A lot of papers use TLA$^+$ in

- OSDI: Symposium on Operating Systems Design and Implementation;
- NSDI: Networked Systems Design and Implementation;
- FM: Formal Methods;
- ...

Follow-up

Post-readings:

- Specifying Systems[1], Lamport (2002)
  Part 1: Getting Started;
  Part 2, Chapter 8: Liveness and Fairness.

opt Practical TLA+: Planning Driven Development, Wayne (2018),
Based in PlusCal.

Related papers:

- Newcombe (2014); Newcombe et al. (2015),
  Describe use of TLA$^+$ at Amazon Web Services.
- Millions of Tiny Databases, Brooker et al. (2020),
  TLA$^+$ and PlusCal are in use at AWS in 2020 as well.
- TLA+ Model Checking Made Symbolic, Konnov et al. (2019),
  Presents Apalache – symbolic model checker for TLA$^+$.

[1] *https://lamport.azurewebsites.net/tla/book-21-07-04.pdf*

Outline

Structure of the TLA$^+$ Language

Example: Round Synchronization

Example: Connection Locks

Example: Brute-force Problems

TLA$^+$ Syntax in More Details

Common Modelling Tricks

Follow-up

References

References

Abadi, M. and Lamport, L. (1994). Open systems in tla. In *Proceedings of the Thirteenth Annual ACM Symposium on Principles of Distributed Computing*, pages 81–90.

Alfatafta, M., Alkhatib, B., Alquraan, A., and Al-Kiswany, S. (2020). Toward a generic fault tolerance technique for partial network partitioning. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 351–368. USENIX Association.

Beers, R. (2008). Pre-rtl formal verification: An intel experience. In *Proceedings of the 45th Annual Design Automation Conference*, DAC '08, page 806–811, New York, NY, USA. Association for Computing Machinery.

Brooker, M., Chen, T., and Ping, F. (2020). Millions of tiny databases. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 463–478, Santa Clara, CA. USENIX Association.

Davis, A. J. J., Hirschhorn, M., and Schvimer, J. (2020). Extreme modelling in practice. *Proc. VLDB Endow.*, 13(9):1346–1358.

Konnov, I., Kukovec, J., and Tran, T.-H. (2019). Tla+ model checking made symbolic. *Proc. ACM Program. Lang.*, 3(OOPSLA).

Lamport, L. (1994). The temporal logic of actions. *ACM Trans. Program. Lang. Syst.*, 16(3):872–923.

Lamport, L. (2002). *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley Longman Publishing Co., Inc., USA.

References (cont.)

Newcombe, C. (2014). Why amazon chose tla+. In Ait Ameur, Y. and Schewe, K.-D., editors, *Abstract State Machines, Alloy, B, TLA, VDM, and Z*, pages 25–39, Berlin, Heidelberg. Springer Berlin Heidelberg.

Newcombe, C., Rath, T., Zhang, F., Munteanu, B., Brooker, M., and Deardeuff, M. (2015). How amazon web services uses formal methods. *Commun. ACM*, 58(4):66–73.

Wayne, H. (2018). *Practical TLA+: Planning Driven Development*. Apress.